Biblioteca para Manipulação de Grafos — Parte 2

Caminhos Mínimos, Filas de Prioridade e Redes de Colaboração

Arthur Sampaio Carlos Bruno

COS 242 — Teoria dos Grafos
PESC / COPPE / UFRJ

Novembro de 2026

Escopo da Parte 2 e Evolução da Arquitetura

Objetivo: Estender a biblioteca em C++17 (Parte 1) para grafos não-direcionados ponderados, cálculo de distâncias e caminhos mínimos.

Decisões de Arquitetura:

- **Consolidação em CSR (`CSRGraph`):** Remoção do módulo legado `BitMatrixGraph` ($\Theta(V^2)$ bits inviável para $V \geq 10^6$).
- **Estrutura *Structure of Arrays* (SoA):**
  - `offsets_` ($n+2$ posições, `size_t`)
  - `edges_` ($2m$ posições, `int`)
  - `weights_` ($2m$ posições, `float`) — alinhado posição a posição com `edges_`
- **Custo exato em RAM:** $8n + 16m + 16$ bytes (apenas 16 bytes por aresta não-direcionada).

Otimização de Memória: Leitura CSR em Duas Passagens

Problema na abordagem clássica de 1 passagem:

- Armazenar um vetor temporário `temp_edges` ($\mathcal{O}(E)$) junto com o CSR final dobrava o pico de memória RAM durante o carregamento.

Solução implementada (`CSRGraph` em 2 passagens no arquivo):

1. **Passagem 1:** Conta o grau de cada vértice, valida arestas e detecta pesos negativos (`has_negative_weight_`) sem guardar arestas em RAM.
2. **Alocação:** Calcula somas prefixadas em `offsets_` e aloca `edges_` e `weights_` no tamanho exato $2m$.
3. **Passagem 2:** Faz `seekg(0)` e preenche diretamente os vetores finais do CSR.

Resultado: Pico de memória estritamente $\mathcal{O}(V + E)$ (93,5 MB no `W_3`; 476,8 MB no `W_4`; 648,5 MB no `W_5`).

Algoritmos de Caminho Mínimo e Decisões de Projeto

- **Grafos sem pesos:** Busca em Largura (`bfs`) em $\mathcal{O}(V + E)$.
- **Bloqueio de pesos negativos em $\mathcal{O}(1)$:**
  - Antes de executar o Dijkstra, consulta `hasNegativeWeight()`.
  - Lança `std::invalid_argument` (testado com a aresta $-9{,}5$ da Figura 1).
- **Precisão numérica:** Pesos armazenados em `float` no CSR (economia de RAM), mas distâncias acumuladas em `double` para evitar perda de precisão em caminhos longos.
- **Parada antecipada (`dijkstraToTargets`):**
  - Na consulta do Item 1 ($10 \to \{20, 30, 40, 50, 60\}$), encerra a busca assim que os 5 alvos são extraídos da fila.
  - No Item 2 (benchmark), executa a busca completa para todos os vértices.

Fila de Prioridade: Vetor vs. Heap Binário Indexado

1. **Modo Vetor (`VectorDistanceStore`)**
   - Mantém distâncias em arranjo linear + flags `visited_` e `queued_`.
   - Contador `remaining_active_` evita varredura final inútil quando a componente conexa termina.
   - `extractMin`: busca linear em $\mathcal{O}(V)$.
   - **Complexidade:**

$$\mathcal{O}(V_r \cdot V + E_r) \subseteq \mathcal{O}(V^2 + E)$$

2. **Modo Heap (`IndexedMinHeap`)**
   - Implementado do zero (sem `std::priority_queue` e sem duplicatas).
   - Três vetores sincronizados: `heap_`, `pos_` (índice inverso em $\mathcal{O}(1)$) e `key_`.
   - `decreaseKey` via `siftUp` em $\mathcal{O}(\log V)$ com no máximo V elementos ativos.
   - **Complexidade:**

$$\mathcal{O}((V + E) \log V)$$

Estudo de Caso 1: Distâncias e Caminhos Mínimos (Origem = 10)

Grafo	Destino	Distância	Saltos	Prefixo / Resumo do Caminho Mínimo
W_3 (10^5)	20	0,8000	18	10 -> 2858 -> 35293 -> 3905 -> 24445
W_3 (10^5)	30	0,8900	14	10 -> 2858 -> 35293 -> 96094 -> ...
W_3 (10^5)	40	0,8400	10	10 -> 2858 -> 35293 -> 3905 -> 24445
W_3 (10^5)	50	0,8700	9	10 -> 56663 -> 84437 -> 19040 -> ...
W_3 (10^5)	60	0,9700	16	10 -> 2858 -> 35293 -> 3905 -> 24445
W_4 (10^6)	20 / 30 / 40	2,66 / 2,02 / 2,52	13 / 17 / 15	Tronco comum (30..60): 10 -> 873980 ->
W_4 (10^6)	50 / 60	2,69 / 2,41	18 / 19	Caminhos completos reportados no relatório
W_5 (10^7)	20 / 30 / 40	17,34 / 17,24 / 17,37	13 / 18 / 14	Tronco comum (40, 50): 10 -> 1299954
W_5 (10^7)	50 / 60	15,43 / 17,99	12 / 16	Menor distância até 50 (15,4300 em 12 saltos

Observações:

- Estrutura de árvore geradora de caminhos mínimos evidente nos prefixos compartilhados.
- Menor distância ponderada $\neq$ menor número de arestas (ex.: no W_3, destino 20 tem custo 0,80 com 18 saltos; destino 50 tem custo 0,87 com apenas 9 saltos).

Estudo de Caso 1: Desempenho Heap vs. Vetor e Extrapolação

Grafo	Vértices (V)	Arestas (E)	Tempo Heap	Tempo Vetor	Razão (Vetor/Heap)
`grafo_W_3`	100.000	$6{,}1 \times 10^6$	0,1372 s	16,8956 s	123,1×
`grafo_W_4`	1.000.000	$31{,}0 \times 10^6$	1,6846 s	1.681,7468 s (28 min)	998,3×
`grafo_W_5`	10.000.000	$40{,}0 \times 10^6$	24,8586 s	$\approx$ 46,7 horas*	$\approx$ 6.760×*
`grafo_W_6`	10.000.000	—	—	$\approx$ 46,7 horas*	—
`grafo_W_7`	50.000.000	—	—	$\approx$ 48,7 dias*	—

*Estimativa por extrapolação quadrática ($k = 1$ busca completa; mesmas origens sorteadas).

Análise Experimental vs. Assintótica:

- **Confirmação do $\mathcal{O}(V^2)$ no Vetor:** Ao multiplicar V por 10× (`W_3` → `W_4`), a teoria prevê $10^2 = 100\times$. Medido: $1.681{,}75/16{,}90 = \mathbf{99{,}54}\times$.
- **Barreira de tempo nos grafos gigantes:** Em `W_5` ($V = 10^7$), uma única busca vetorial leva $\approx$ 47 h; em `W_7` ($V = 5 \times 10^7$), $\approx$ 49 dias ($>$ 52 dias acumulados).
- **Escalabilidade do Heap Indexado:** Resolve 10 milhões de vértices (`W_5`) em apenas **24,86** s.

Estudo de Caso 2: Rede de Colaboração Científica (DBLP)

Grafo DBLP: $V = 722.385$ pesquisadores, $E = 2.272.543$ coautorias ($w(u, v) = 1/\text{nž artigos}$).
Origem: `Edsger W. Dijkstra [ID 2722]`

Pesquisador Alvo	Distância	Saltos	Caminho Mínimo de Coautoria
Alan M. Turing	∞	—	*Inalcançável (fora da componente conexa de Dijkstra)*
J. B. Kruskal	3,480368	8	`Dijkstra -> J. R. Rice -> D. C. Marinescu -> H. J. Siegel -> E. K. P. Chong -> N. B. Shroff -> R. Srikant -> A. G. Greenberg -> Kruskal`
Jon M. Kleinberg	2,706994	9	`Dijkstra -> A. van Gasteren -> G. Tel -> H. Bodlaender -> D. Thilikos -> P. Ragde -> A. Wigderson -> E. Upfal -> P. Raghavan -> Kleinberg`
Éva Tardos	2,753514	11	`Dijkstra -> A. van Gasteren -> G. Tel -> H. Bodlaender -> J. van Leeuwen -> M. Overmars -> M. Sharir -> H. Kaplan -> R. E. Tarjan -> A. Goldberg -> S. Plotkin -> Tardos`
Daniel R. Figueiredo	2,942831	8	`Dijkstra -> J. R. Rice -> D. C. Marinescu -> C. Lin -> B. Li -> Y. T. Hou -> Z.-L. Zhang ->`

Interpretação da Rede DBLP e Conclusões

Destaques da Rede de Colaboração:

- **Alan M. Turing (∞):** Publicações históricas sem coautoria conectada à componente gigante moderna do DBLP.
- **Duas pontes principais a partir de Dijkstra:**
  - *Ramo Holandês / Teoria* (`van Gasteren -> Tel -> Bodlaender`): leva a **Kleinberg** e **Éva Tardos**.
  - *Ramo Sistemas / Redes* (`John R. Rice -> Marinescu`): leva a **Kruskal** e ao Prof. **Daniel R. Figueiredo** (via `Zhi-Li Zhang -> Donald F. Towsley`).

Conclusão Geral:

- A leitura CSR em duas passagens reduziu drasticamente o pico de memória ($\mathcal{O}(V + E)$ estrito).
- O `IndexedMinHeap` ($\mathcal{O}((V + E) \log V)$) foi até $\mathbf{1.000}\times$ mais rápido que o vetor no `W_4` e viabilizou buscas em 10^7 vértices em 24,8 s, enquanto o modo vetor ($\mathcal{O}(V^2)$) torna-se proibitivo.